

Rozgar

Understand the job platform, test every major part, and explain your engineering decisions with confidence.

Honest status: The website and API are deployed and a three-source job-discovery beta works. The original eight-week roadmap is **not fully satisfied**: source-count targets, a running production scheduler, real notification delivery, and real-user impact evidence remain open.

Prepared 15 September 2026 from repository commit `87fd0d8` and public smoke checks. This guide contains no environment-variable values or private credentials.

1. What problem are we solving?

People looking for remote work must search many job boards, compare inconsistent listings, and remember which roles they liked. Rozgar collects jobs from several sources into one searchable website. A visitor can filter jobs, read details, go to the employer to apply, and keep a private shortlist in their browser. An account holder can save a search so the system can later match new jobs to that interest.

The lead's program asks an intern to own the full outcome, from collection to website to deployment. Its longer-term target was a platform with at least 50 sources, autonomous data collection, and real user notifications. The current repository is a smaller but working beta. Present it that way.

Job boards → Python scrapers → PostgreSQL → FastAPI → Next.js website → job seeker

Optional side path: saved search → separate scheduler → Resend email, once configured

Words you should know

Word	Beginner explanation
Frontend	The website the user sees and clicks.
Backend / API	The server that receives requests, checks rules and identity, and reads or writes data. An API endpoint is a URL with a specific action.
Scraper	Code that fetches a source's job data and converts it into a shared format.
Database	Durable storage. PostgreSQL holds jobs, saved searches, scrape runs, and alert-delivery records.
Cache	A short-lived copy of a result. Redis can accelerate repeated API searches; the browser also briefly caches public results.
Scheduler / worker	A separate running process that starts scraping and alert checks automatically. Deploying the API does not start it.
CI / migration	CI runs checks on code pushes. A database migration changes the stored schema in a controlled way.

2. What we used and why

Part / service	Job in Rozgar	Current state
Next.js, React, TypeScript, Tailwind CSS	Responsive pages, controls, job feed, detail pages, shortlist and saved-search UI.	Deployed on Vercel as <code>codeaza1/jobsi</code> .
FastAPI, Python, Pydantic	HTTP endpoints, request validation, scrapers, and authenticated operations.	Deployed on Vercel as <code>codeaza1/jobs</code> .
PostgreSQL; SQLAlchemy; Alembic	Durable records, queries, indexes, and schema upgrades.	Production database responds to readiness and job queries.

RemoteOK, Arbeitnow, Jobicy	Three implemented source adapters.	Registered in code; current feed contains source data.
Clerk	Sign-in, signed session tokens, and verified account email.	Development-instance settings were previously used for an authenticated test; repeat the user journey yourself.
Redis	Optional server-side search cache; PostgreSQL remains usable if it fails.	Code implemented; current production readiness reports <code>cache: disabled</code> .
APScheduler + Resend	Start source runs and queue/deliver matching emails.	Code implemented; no production worker or verified sender/delivery evidence. Email alerts are paused.
Sentry	Capture and scrub unexpected errors.	Integration code exists; receipt of a real event in a Sentry account is unverified.
GitHub Actions, Playwright, pytest, Vitest	Automated quality checks and browser tests.	Final combined-repository CI run passed.
GitHub + Vercel	One repository; two projects with different root folders and domains.	Both public deployments were ready from the same commit.

Frontend: jobsi-ten.vercel.app. API: jobs-codeaza1.vercel.app. All Git commits now go to the combined `Rozgar` repository; Vercel builds `Frontend` and `Backend` separately.

3. How the system works

1. A source adapter fetches jobs. `BaseScraper` spaces requests, retries temporary network/server errors, and validates a common `NormalizedJob` shape. Invalid individual jobs are skipped rather than throwing away a whole batch.
2. The pipeline writes normalized jobs and keeps raw source information. It removes tracking parameters from application URLs and uses a URL fingerprint to prevent new copies with the same application link. Similar titles with different links may remain separate.
3. FastAPI reads filtered jobs from PostgreSQL. The feed requests `summary=true` so large descriptions are not fetched for cards. The full description is requested for one job detail page.
4. The browser sends search/filter parameters to `/jobs`. The API validates them, returns newest jobs first, and supports `before_id` cursor paging. The website shows loading, empty, and retry states.
5. A shortlist is stored in that browser's `localStorage`. It needs no account, survives a reload on the same browser, and does not automatically sync to another phone or browser.
6. For saved searches, Clerk signs the user in. The frontend passes a bearer token; the API verifies it and derives the user ID. The API obtains the user's verified primary email from Clerk. Each user can access only their own searches.
7. If a separate scheduler and email credentials are running, new matching jobs become durable alert-delivery rows. A unique search/job pair and a stable provider idempotency key reduce duplicate messages. Email delivery is paused in the current public setup.

Design principle: PostgreSQL is the source of truth. Redis is an accelerator, not the only place where jobs live. The API is the authorization boundary; browser-supplied user IDs and email addresses cannot decide ownership.

4. Coverage against the lead's eight-week roadmap

Do not say “all eight weeks are complete.” There is no honest single completion percentage because the milestones differ in size. The table shows exactly what is covered.

Week	Roadmap outcome	Evidence and remaining work
1	One end-to-end app	Core loop achieved: deployed website → API → PostgreSQL → job data.
2	Reusable source system and 10 sources	Partial: common scraper/normalized job shape exists; only 3 source adapters, not 10.
3	50+ reliable sources and clean deduplication	Partial: retries, delay, run status and URL-based deduplication exist; 50+ sources and hard-site browser automation do not.
4	Autonomous scheduled engine on Codeaza server	Partial: scheduler and Docker code exist; current Vercel API does not start a worker and Codeaza-server operation is unverified.
5	Mobile job feed, filters, scrolling, pages	Implemented and deployed: responsive feed, filters, job details, load-more/scroll, light/dark mode, shortlist; live keyword search checked.
6	Accounts, saved searches, real email/WhatsApp alerts	Partial: Clerk-based searches and an email engine exist; public email is paused, real delivery is unverified, and WhatsApp is absent.
7	Speed, tests, CI, monitoring, dashboard	Mostly implemented: code for Redis caching, Sentry, admin health, tests and CI. Local warm-cache p95 measured 11.89 ms versus 39.43 ms uncached on 10,000 synthetic jobs. Production Redis and real Sentry/admin checks remain unverified.
8	Live users, metric improvement and defense	Partial: website/API live, clean repo, runbook, benchmark and 20-minute demo script. No demonstrated real-user adoption or production p95 data; recorded p95 is a local benchmark.

The source-count numbers and real-alert requirement come from the supplied lead roadmap. The p95 figures come from `docs/week-8-release.md`; they exclude public network latency. A working beta is valuable, but it is not proof of the 50-source and real-user release.

5. How to test the public website, step by step

Start here. These tests need only your browser. Keep this PDF beside the website. Wait until job cards appear before typing; an automated test that filled the input before hydration did not retain the text.

Test 1 — The site and API are alive.

Open [the website](#). You should see the Rozgar home page and job cards. In new tabs open [API live](#) and [API ready](#). Expected: `{"status": "ok"}` for live and `database: "ok"` for ready. `cache: "disabled"` is the current production setting, not a broken database. ☐ Pass ☐ Fail

Test 2 — Search and filter real jobs.

Once cards appear, type `Python` in “Job title, skill, or company.” Expected: related results; the live request `/jobs?keyword=Python...` previously returned HTTP 200 with 9 results. Try Location, “Remote only,” “Salary listed only,” and a minimum salary. Results should narrow. A combination can reasonably show “No matching jobs yet.” Use “Clear filters” to reset. ☐ Pass ☐ Fail

Test 3 — Feed navigation and job details.

Try a “Python” quick-search button or a career path. If “Load more jobs” appears, click or scroll near the end and check that more cards arrive without replacing previous cards. Open a job title. Expected: detail page and an “Apply on employer” link leading to the original source; do not submit a real application just for the test. Some result sets have fewer than 20 jobs and therefore no extra page. ☐ Pass ☐ Fail

Test 4 — Shortlist.

Click the bookmark on one job card. Open “Shortlist,” reload that page, and confirm the saved job is still there. Remove it and confirm it disappears. Repeat in a private/incognito window: it will be a separate list, because the feature uses that browser’s storage. ☐ Pass ☐ Fail

Test 5 — Interface controls.

Toggle dark mode and reload; the chosen theme should remain. Switch grid/list view. Resize the browser to a narrow phone width, and check that text and controls fit without horizontal scrolling. Press `/` while not typing in another field; search should receive focus. ☐ Pass ☐ Fail

Test 6 — Failure handling without changing production.

If the API temporarily fails, the feed should show an understandable message and “Try again.” The repository’s Playwright tests simulate an HTTP 503 and verify this behavior; do not deliberately shut down the production database to test it. ☐ Automated test ☐ Not attempted

6. Test accounts and saved searches

Use your own test account. The current project has used Clerk development-instance settings, so do not treat this as a verified public launch. If Clerk rejects the domain, check the matching frontend/backend Clerk settings in Vercel before claiming the feature broken.

Test 7 — Sign in and create.

Click “Sign in.” Complete Clerk’s flow with an address you control and verify it if prompted. Return to the job feed, type a keyword such as `Python`, then click “Save search.” Expected: a success message and an item under “Saved searches.” If you click without a keyword, the page should ask you to enter one. The saved-search page currently says email alerts are paused. ☐ Pass ☐ Fail

Test 8 — Ownership and deletion.

Reload “Saved searches”; the item should remain. Delete that test search; it should disappear. Sign out and open “Saved searches”: you should see a sign-in invitation, not the previous account's list. A different signed-in account should not see the first account's list. The backend's ownership test checks that another user also cannot delete it by ID. ☐ Pass ☐ Fail

Test 9 — Duplicate search.

Save the same keyword and same filters twice while signed in. The backend reuses the existing active record. Change a filter and save again; that is a different search. A user can keep at most 20 active searches. ☐ Pass ☐ Fail

Do not claim a received alert from these tests. Saving a search works independently of email. Public email delivery needs a verified sender, Resend key, backend Clerk secret, and a separate scheduler. WhatsApp delivery is not implemented.

7. Test the API without coding

Open [FastAPI's interactive docs](#). A blue `GET` reads data; a `POST` starts or creates something; a `DELETE` removes/deactivates it. Expand an endpoint, select “Try it out,” enter parameters, then “Execute.”

Endpoint to inspect	What to try and what you should see
<code>GET /jobs</code>	Use <code>keyword=Python</code> , <code>limit=5</code> , <code>summary=true</code> . Expect HTTP 200 and a JSON list. For summary results, description is omitted. Try <code>remote_only=true</code> ; a valid but empty list is possible.
<code>GET /jobs/{job_id}</code>	Copy an ID from a result. Expect one job with an application URL. An invalid ID should return 404.
<code>GET /health/live</code> and <code>/health/ready</code>	Expect 200. Ready checks the database; a 503 means that dependency failed.
<code>GET /saved-searches/</code> or <code>/health</code> while signed out	These are protected. Without a valid bearer token, expect 401. Do not paste your login token into a shared screenshot or PDF.
<code>POST /scrape/{source}</code>	Requires a private <code>x-api-key</code> and starts a real external fetch. Test it only in a local/test environment you control, with permission to call that source; do not treat it as a casual public button.

HTTP 200 means the request worked. HTTP 401 means unauthenticated; 403 means signed in but not allowed; 404 means item not found; 422 means invalid input; 503 means a required dependency is unavailable.

8. Test locally and run automated checks

Why test locally? It lets you inspect the backend, scheduler, Redis, and migrations without changing the live site. Start with automated checks if PostgreSQL/Redis are already available. Commands below use Windows PowerShell, as in the repository README.

1. Open a terminal at the `Rozgar` repository root. Run `git status`. Do not edit or share `Backend/.env`; it contains private settings. Use `.env.example` to learn variable names.
2. For a new environment, install Python 3.11, Node.js 24 and PostgreSQL. Redis is optional for normal reads but needed for the cache test. The example local database uses port 55432; edit your local environment only if your services use other ports.
3. Open one terminal for the API and another for the website:

```
cd Backend
python -m venv venv # Only if venv does not already exist
.\venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.\venv\Scripts\python.exe -m alembic upgrade head
.\venv\Scripts\python.exe -m uvicorn app.main:app --reload

# In another terminal, from the Rozgar root:
cd Frontend
npm ci
npm run dev
```

Before starting, keep `Backend/.env` and `Frontend/.env.local` aligned: `DATABASE_URL` points to your PostgreSQL; `NEXT_PUBLIC_API_URL=http://127.0.0.1:8000`; local CORS/Clerk allowed origins include `http://localhost:3000`. Open [local API docs](#) and [local website](#). If a port is already in use, stop the old process or choose a consistent new port.

Automated quality checks

```
# Backend terminal, with local test PostgreSQL/Redis running:
.\venv\Scripts\python.exe -m ruff check app database.py alembic tests scripts
.\venv\Scripts\python.exe -m pytest -q
.\venv\Scripts\python.exe scripts/benchmark_search.py

# Frontend terminal:
npm run lint
npm run typecheck
npm test
npm run build
npm run test:e2e
```

Playwright's browser test uses a production build on local port 3100 and mocked job responses; install Chromium with `npx playwright install chromium` if prompted. These tests cover UI behavior, not proof that a real external scraper or email provider works. GitHub Actions performs the broader CI checks automatically after a push.

Optional operator tests, after you have a local test database

Part	Safe test and expected result
------	-------------------------------

Scrapers and run tracking	With your private <code>SCRAPE_SECRET_KEY</code> , invoke one local <code>POST /scrape/arbeithnow</code> in API docs. Expect <code>status: ok</code> with added/skipped counts, or a recorded <code>failed</code> result if the external source changed. Repeat: identical application URLs should be skipped. Do not publish the key.
Scheduler	In a third <code>Backend</code> terminal, run <code>.\venv\Scripts\python.exe -m app.run_scheduler</code> . It starts registered sources and checks alerts every minute. Stop with <code>Ctrl+C</code> . Use a disposable/test database if you are investigating new jobs or alert matching.
Redis cache	Set a local <code>REDIS_URL</code> to a running Redis. Make the same <code>GET /jobs</code> request twice: expect <code>X-Cache: MISS</code> then <code>HIT</code> . Without Redis or while it fails, expect <code>BYPASS</code> and still get database jobs.
Admin health	Set your Clerk user ID in local <code>ADMIN_USER_IDS</code> and sign in. Inspect <code>/health</code> with a valid token. It reports each source's latest run, total jobs, alert counts and monitoring flags. A non-admin must be refused.
Real email and Sentry	Only after configuring your own test Resend sender, Clerk settings, worker, and Sentry accounts should you run controlled end-to-end delivery and event-receipt tests. Check one matching email, no duplicate after a repeated run, and scrubbed Sentry data. These are external acceptance gates, not yet documented as passed.

9. Reading results and fixing the common first problems

What you see	First check
Website shows cards forever loading or retry message	Open API <code>/health/ready</code> . Check the frontend's <code>NEXT_PUBLIC_API_URL</code> and the backend's CORS origins; rebuild the frontend after changing a public build variable.
API ready returns 503	Check <code>DATABASE_URL</code> , PostgreSQL reachability, and whether <code>alembic upgrade head</code> has run. Avoid printing the database URL in a screenshot.
Search returns no jobs	Clear filters and search a broader word. An empty result can be correct. If all searches are empty, inspect database job count and source run history.
Saved-search button fails	Check sign-in, a verified email, matching Clerk frontend/backend instance keys and authorized origins. A keyword is mandatory.
Shortlist vanishes in another browser	This is expected: shortlist lives in local browser storage. Use the same browser/profile for the persistence test.
Saved search exists but no email arrives	Current public delivery is paused. Resend credentials, verified sender, active scheduler and a newly matching job are needed.
Vercel build fails	Inspect the failing project's build log. The frontend root is <code>Frontend</code> , backend root is <code>Backend</code> ; the backend also has a Vercel-readable root Python requirements file. Use the previous successful deployment if a bad production release needs rollback.

10. A 20-minute demo you can give your lead

- 0–3 minutes:** Explain the user problem and honestly say “three-source beta.” Show the live URLs.
- 3–8 minutes:** Search `Python`, filter, open a job, shortlist it, reload, switch dark mode and narrow the browser width.

3. **8–11 minutes:** Sign in, save a keyword search, show its persistence and delete it. Explain why email is paused.
4. **11–15 minutes:** Point to scraper → normalized record → URL deduplication → PostgreSQL → API. Show an API JSON result and readiness response.
5. **15–18 minutes:** Show the GitHub CI result and local p95 benchmark. Say clearly that these numbers are synthetic local measurements, not production user latency.
6. **18–20 minutes:** Name the gaps and propose next evidence: 10 and then 50 approved source adapters, an always-on scheduler, one real verified alert and a week of production p95/usage data.

11. Questions your lead may ask in an interview

Practice answering in your own words. The short answers below are examples to understand, not lines to memorize. If you cannot explain an answer, open the linked source file and trace the flow.

1. What is Rozgar's goal?

Bring remote jobs from several boards into one searchable feed, let a visitor save roles, and prepare account-based notifications for new matches.

2. Is the roadmap complete?

No. The live three-source beta and CI are real, but 10/50 source milestones, autonomous production scheduling, real email/WhatsApp delivery, and real-user metric evidence are not complete.

3. Why one GitHub repo but two Vercel projects?

The apps share a product and release commit but use different runtimes. Vercel builds `Frontend` with Next.js and `Backend` with Python, retaining separate domains and environment settings.

4. What happens when a user types a keyword?

React updates filters; the website sends a debounced `/jobs` request; FastAPI validates parameters; SQLAlchemy filters PostgreSQL jobs; the response renders cards.

5. What does `summary=true` improve?

Feed cards omit large descriptions from both the SQL column fetch and returned JSON. The detail page fetches the full job when needed.

6. What is cursor paging?

The next request passes the last displayed job ID as `before_id` and asks for older IDs, instead of repeatedly reading farther with a large offset.

7. How are duplicates detected?

We normalize the application URL, remove common tracking parameters, hash it, and prevent new rows with the same fingerprint. We do not promise perfect semantic deduplication across different links.

8. Why keep raw source data?

It helps investigate parsing mistakes and source changes without losing the original information behind a normalized job.

9. How does a scraper handle temporary failure?

It spaces requests, retries network errors, 429, and server errors with increasing delay, and records a failed source run if the job pipeline cannot finish.

10. Why use PostgreSQL and Redis?

PostgreSQL keeps durable truth. Redis can hold short-lived search responses. If Redis is absent or unavailable, the API bypasses it and reads PostgreSQL.

11. What does `/health/live` prove? What does `ready` prove?

Live proves the API process responds. Ready also checks a database query and reports cache status. Neither proves that all scrapers or emails are healthy.

12. How do saved searches stay private?

The API verifies Clerk's bearer token, derives the user ID itself, obtains the verified email from Clerk, and filters reads/deletes by owner. It does not trust a user ID typed by the browser.

13. Shortlist versus saved search?

Shortlist is a browser-local list of individual jobs, available without sign-in. Saved search is a server-side, account-owned query with filters, intended for matching future jobs.

14. How do we avoid sending the same job alert twice?

The database has one delivery record per search/job pair. Retry sends use a stable Resend idempotency key. Ambiguous old attempts move to review rather than blindly repeating.

15. Why is the scheduler a separate process?

HTTP functions answer requests and may stop between them. Continuous periodic scraping/alert checks require a worker kept running separately; the Vercel API deployment alone is not that worker.

16. What is the measured performance improvement?

In the recorded local 10,000-synthetic-job test, search p95 fell from 39.43 ms uncached to 11.89 ms cached, about 69.8% lower. Production Redis is currently disabled, so do not apply that claim to live users.

17. What do CI and browser tests cover?

CI runs lint, types, backend tests, migration/benchmark checks, build and desktop/mobile browser tests. Browser tests use mocked jobs for repeatability; live source/provider tests remain separate.

18. What would you do when production fails?

Find the failing layer with live/ready, API response and Vercel logs; preserve database state; repair and test, or promote the previous successful project deployment. Do not delete alert-delivery history to roll back code.

19. What would you improve next?

Add approved sources toward 10 then 50, operate one scheduler, finish verified test email and Sentry gates, configure production Redis, and measure one week of real search latency and user activity.

20. What part did you personally build and verify?

Be specific and truthful. Separate work you wrote or changed, work you reviewed, automated checks that passed, and external features you did not verify. A candid answer is stronger than claiming every subsystem is finished.

Follow-up preparation: Your lead may ask you to find an endpoint in `Backend/app/routers`, show the PostgreSQL model in `Backend/app/models.py`, explain a frontend state change in `Frontend/app/page.tsx`, or trace an alert from `Backend/app/scheduler.py` through `Backend/app/alerts.py`. Rehearse those four file paths.

12. Evidence to collect before saying “finished”

Evidence	Your record
Public website and API ready; a live keyword, detail, shortlist, sign-in and saved-search journey	Date, browser, result, screenshot with no token or credentials: _____

Source coverage	Registered and healthy sources: ____ / 10 first target; ____ / 50 final target
Autonomous worker	Where it runs, uptime, last three source runs: _____
One real notification, no duplicate after repeated scrape	Test account, provider message ID, delivery row, date: _____
Real monitoring and performance	Sentry scrubbed event IDs; production p95 sample count/date range: _____
Real users	Approved user count and outcome metric, with dates: _____

Repository references: [README.md](#) , [docs/architecture.md](#) , [docs/deployment.md](#) , [docs/week-7-plan.md](#) , [docs/week-8-release.md](#) , and the supplied lead roadmap. Public checks on 15 September 2026 returned HTTP 200 for the website, API live, API ready and a job summary. Re-check live status when you present; hosted services can change after this guide was written.